Neural MCPF Allocation

Learning the Goal-Allocation Step of Multi-Agent Path Finding

Ofek Yabo | Dayan Badalbaev

Multi-Agent Systems, 2026

Abstract

Multi-agent path finding with multiple goals requires deciding which agent visits which goals before any path can be planned. This allocation sub-problem is a multi-agent traveling salesman problem, and it is the stage that limits how far the solver can scale. The exact RobustMCPF solver handles it with an LKH TSP call, then passes a fixed allocation to conflict-based search (CBS) for collision-free planning, a stage that is comparatively well behaved. We replace only the allocation step with a learned transformer that is size-agnostic: a single set of weights handles any number of agents and goals. It reads two shortest-path distance matrices, one from agents to goals and one between goals, and outputs a distribution over agents for each goal. Since CBS is shared by both pipelines, any difference in the final plan comes from the allocator alone. We evaluate it at problem sizes from 2×2 up to 50×100. Across 28 configurations the collision-free plans it produces are on average 2% longer than the solver's optimal plans and at worst 5% longer, and every allocation it proposed could be planned without collisions. At paper scale it runs 5.9–9.4× faster while remaining within roughly 20% of optimal. We then ask whether map-specialized allocators beat a single generalist. Five models, one trained on all four benchmark maps and four trained on a single map each, are compared on solution quality and wall-time to determine whether map-aware routing is worthwhile. Pushed further, to 220×430 on the four real benchmark maps, the same generalist stays within 9.6% of optimal at 6.6–16.1× speedup. The generalist ties its map specialists on cost everywhere, including the structured maze, and the real risk is not specializing too little but misrouting an instance to the wrong specialist. A second training-data comparison, many regular-size instances against fewer large ones, finds no crossover: the regular-size dataset wins on every model role, on both its own scale and the large one, so a fixed labeling budget is better spent on more instances at a moderate scale than on fewer instances pushed to the scale limit.

1 Introduction

Fleets of autonomous agents such as warehouse robots, delivery drones and search-and-rescue teams all face the same first decision: given a set of targets and a team of agents, who goes where? The choice matters. A poor allocation sends agents across each other's paths, inflates total travel, and creates collisions that a planner then has to resolve. It also has to be made before anything else, since path planning only becomes meaningful once each agent knows its goals.

Formally this is multi-agent path finding (MAPF) with the added requirement that goals be assigned. The assignment is not a balanced one-to-one matching. An agent may profitably take zero, one, or several goals, which makes the structure a multi-agent traveling salesman problem (mTSP). The RobustMCPF solver [1] solves it exactly in two stages. An LKH TSP call [3] produces the optimal allocation together with each agent's visit order, and conflict-based search (CBS) [2] converts that into collision-free paths. The two stages scale very differently. CBS cost depends on how much the agents interfere with each other, whereas the TSP allocation is combinatorial in the number of agents and goals, and it is the stage that degrades first as problems grow.

This work replaces that stage, and only that stage, with a neural network that reads two breadth-first-search distance matrices: D from agents to goals, and G between goals. Both pipelines then plan collision-free paths with the same CBS implementation, so the comparison stays close to like-for-like. The trade is exactness for speed. A network trained by imitation will

not always reproduce the optimum, and the question this report answers is when accepting that is worthwhile.

Contributions.

- **A size-agnostic transformer allocator.** One set of weights serves any (N, M), with no positional embeddings and no padding, so a single model replaces what would otherwise be one network per problem size, and nothing in the architecture bounds how large a problem it will accept (Section 3).
- **Evaluation by true execution cost rather than allocation accuracy.** The two disagree sharply: 64–97% of instances match the solver's cost exactly while only 44–91% match its allocation, because many different allocations are cost-equivalent ties. Judging the model on allocation accuracy alone understates it (Section 4).
- **A design study identifying input representation as the dominant factor.** Adding goal-to-goal distances is worth 14 to 18 points of accuracy, roughly four times what architecture, capacity and training-set size contribute together, which redirected the project from tuning the network to fixing what it reads (Section 5).
- **A characterization of what transfers.** A model trained only on small random grids holds at a 1.019 cost ratio on unseen real maps but collapses to 1.246 once the problem grows, so map geometry largely transfers and problem scale does not. The practical rule is to retrain for size, not for shape (Section 5).
- **A specialist-versus-generalist study at scale.** Four map specialists fail to beat a single generalist on cost, while sending an instance to the wrong specialist costs up to 52%, which recasts map-aware routing from an optimization into a safeguard (Section 6).

2 Background and Problem Setting

2.1 Why replace the allocator

The practical case rests on where the cost sits and how often the problem gets solved. In a warehouse or a delivery fleet, allocation is re-solved continuously as orders arrive and agents finish their tasks, and every re-solve pays the combinatorial cost again. An exact allocator is an excellent choice for a single offline instance and a progressively worse one as throughput requirements rise. A learned allocator has the opposite profile. Training is expensive and paid once, while inference is a single forward pass through the network, a fixed sequence of matrix multiplications with no search involved, whose cost grows far more slowly than the solver's. What is given up is quality. The report measures that loss in the only unit that matters to a user of the system, the total number of steps the agents actually walk, counted after collisions have been resolved. It then identifies where the exchange pays off.

2.2 Problem formulation

We work in basic MAPF: a 4-connected grid with static obstacles, deterministic movement, no orientation and no action delays. An instance consists of N agents at distinct start cells and M goal cells. Every goal is visited exactly once by exactly one agent, and agents are otherwise unconstrained. The objective is min-sum, minimizing total path length, subject to the paths being collision-free.

That freedom is what makes this an mTSP and not an assignment problem. Figure 1 shows the canonical case of two agents near each other and two distant goals. Splitting the goals costs 14, while giving both to one agent costs 7. The optimum is unbalanced.

Write $Y \in \{0,1\}^{N \times M}$ for the allocation: one row per agent and one column per goal, so Y has N rows and M columns, and $Y_{ij} = 1$ exactly when agent i visits goal j. Reading down column j answers "which agent takes this goal", and that column holds exactly one 1, because every goal is visited once by one agent. Reading across row i answers "which goals does this agent get", and that row may hold any number of 1s, including none, because an agent is free to take several goals or sit idle. Columns therefore sum to one while rows are unconstrained.

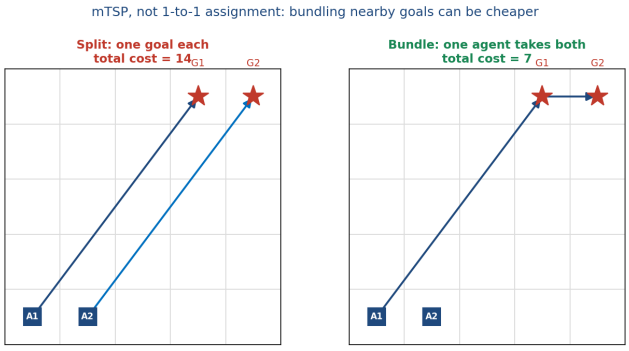


Figure 1: Goal allocation is an mTSP, not a balanced assignment. Bundling both goals onto one agent (cost 7) beats splitting them (cost 14).

The same shape runs through the whole model. The input D and the output P are both $N \times M$ with agents on the rows and goals on the columns, so all three matrices line up entry for entry, while G is $M \times M$ because it relates goals to each other rather than agents to goals. The model therefore has to produce a column-wise distribution, giving for each goal a distribution over agents, instead of a doubly-stochastic or permutation-like output. This rules out Sinkhorn-style balanced-assignment layers, which impose a row constraint the problem does not have.

2.3 The exact baseline

RobustMCPF in BasicMAPF mode serves as both ground truth and comparison point. It calls LKH to solve the underlying TSP, which yields k-best allocations along with each agent's visit order, and then runs CBS to resolve conflicts. We take the cost of the final collision-free plan as the definitive quality measure. The same solver generates the training labels: every training instance is solved exactly, and its allocation becomes the supervision target.

2.4 Related work

Learning to solve combinatorial problems has a substantial literature. Pointer Networks [9] output permutations of their input, reinforcement-learning variants scaled the idea to routing [10], and attention models became the standard backbone for TSP and VRP [11]; [12] surveys the area. Within MAPF, CBS [2] defines the exact-planning baseline, benchmarks and terminology come from [4, 5], and learned policies such as PRIMAL [8] target path planning, not allocation. Combined target assignment and planning was formalized by [6]. Closest to our final study is algorithm selection for MAPF [7], which asks the same underlying question of whether instance-specific specialization beats one general method. We differ from the routing literature in learning only the allocation and leaving an exact planner downstream, which makes the comparison end-to-end and unambiguous.

3 Method: The Learned Allocator

3.1 Overview

The solver runs LKH followed by CBS. Ours runs a forward pass through the network, then goal-visit ordering, then CBS (Figure 2). Both pipelines plan collision-free paths with the same CBS implementation. They differ in how the visit order within each agent's goals is obtained: LKH returns it as part of its tour, whereas our pipeline computes it separately once the allocation is fixed.

3.2 Input representation

The network never sees the grid. It sees two matrices of BFS shortest-path distances that respect walls: $D \in \mathbb{R}^{N \times M}$ from each agent to each goal, and $G \in \mathbb{R}^{M \times M}$ between goals (Figure 3). Both are divided (normalized) by $(W-1)+(H-1)$, where W and H are the grid's width and height in cells. This is the longest shortest path on an open grid of that size, so it puts distances from

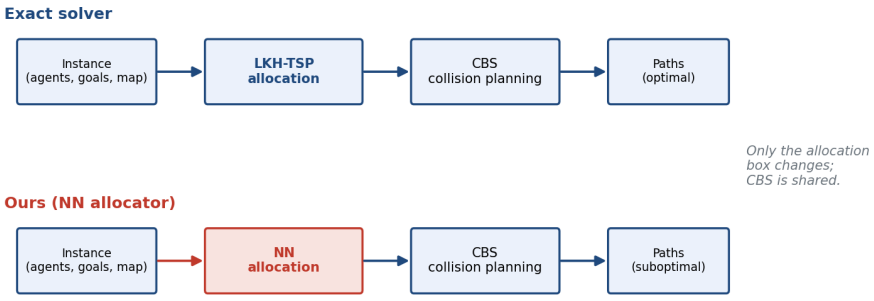


Figure 2: Only the allocation box changes. CBS is shared, so any difference in execution cost or wall-time can be attributed to the allocator.

grids of different sizes on a common scale. Walls can force a detour that exceeds it, so it is a scale factor and not a strict bound.

Using distances instead of coordinates is a deliberate choice. Tour cost is a function of distances, and distances abstract away the map, so the representation transfers across grid sizes and geometries. D tells each agent how far each goal is. G carries the information D cannot express, which is which goals lie near each other and should therefore be bundled onto one agent. Section 5 shows that G is the single most valuable design decision in the project.

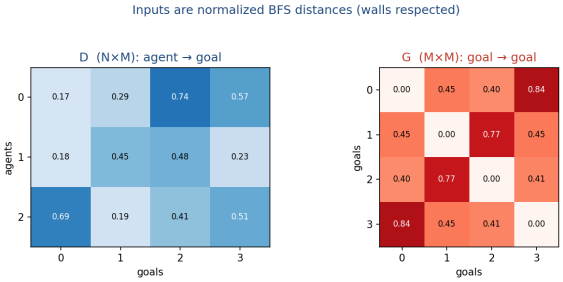


Figure 3: The two inputs for one instance: agent-to-goal distances D and goal-to-goal distances G, both normalized BFS distances with walls respected.

3.3 Handling any number of agents and goals

A conventional network fixes its input dimension, which would force one model per (N, M) pair. Ours does not, and the mechanism is worth setting out in full (Figure 4).

Tokenization. Each scalar D_{ij} is projected independently to a d-dimensional vector by a shared Linear$(1 \to d)$, producing an $N \times M$ grid of tokens. Because the projection is shared, the parameter count does not depend on N or M.

Goal context. Goal j has a row of distances to every other goal, G_{j1} through G_{jM}. Each of those numbers is passed through its own shared Linear$(1 \to d)$, turning it into a d-dimensional vector, and the resulting vectors are then added together elementwise to give a single d-dimensional summary for goal j. Embedding before adding matters, because it lets the network retain more than one property of the distances, and because the sum of any number of d-vectors is still a d-vector, M can be anything. That summary is added to each of the N tokens in goal j's column, which are d-dimensional as well, so every token concerning goal j also carries information about how that goal sits relative to the rest.

Attention. Each block applies row attention, in which an agent attends over its own goals and learns how they relate to each other, then column attention, in which a goal attends over the candidate agents and learns which is best placed to take it, then a feed-forward layer. Blocks are stacked L deep. Attention operates on variable-length sequences by construction.

No positional embeddings. Agents and goals are unordered sets in an mTSP, so there is no meaningful "agent 1". Leaving out positional embeddings makes the network permutation-equivariant in both axes and removes the last size-dependent component, which is why the same weights can process a 2×2 instance and a 50×100 one. The architecture therefore places no upper bound on N or M.

Output. A shared Linear($d \to 1$) reduces each token to a single logit, giving an $N \times M$ logit matrix. A softmax down each column yields P, where P_{ij} is the probability that agent i visits goal j and $\sum_i P_{ij} = 1$. This is exactly the structure Section 2.2 requires. Taking the arg-max of each column decodes the binary allocation.

Mixed-size training. Instances of different shapes cannot be stacked into one rectangular array, which is what a GPU needs in order to process many at once, so a `MixedSizeBatchSampler` builds shape-homogeneous batches. Each batch holds a single (N, M) while consecutive batches differ, which trains one model jointly across many configurations without padding or masking.

A worked example. Take N=3 agents, M=4 goals and embedding width d=128. Here D is 3×4 and G is 4×4. Tokenization turns each of the 12 entries of D into a 128-vector, giving a 3×4 grid of tokens. For goal 2, the four numbers G_{21} through G_{24} each become a 128-vector and are summed into one 128-vector, which is added to all three tokens in column 2. Row attention then runs over sequences of length 4, once per agent; column attention runs over sequences of length 3, once per goal. The head reduces every token to a single number, giving a 3×4 logit matrix, and a softmax down each of the four columns gives each goal a distribution over the three agents. Now change the instance to N=180 and M=350: the token grid becomes 180×350, attention runs over sequences of length 350 and 180 instead of 4 and 3, and every weight in the network is the same one used for the 3×4 case.

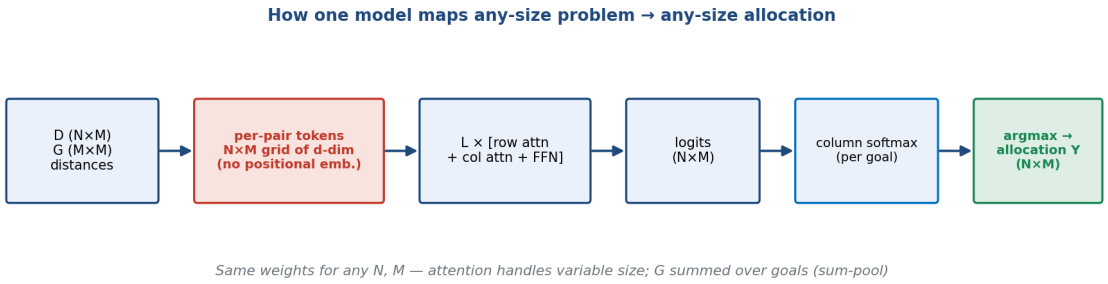


Figure 4: The size-agnostic path from an instance to an allocation. D and G become an $N \times M$ token grid, row/column attention blocks process it, a shared head produces $N \times M$ logits, a column softmax gives a per-goal distribution over agents, and arg-max decodes the allocation. No stage depends on the values of N or M.

3.4 Loss

Training minimizes

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{MinSum}}, \qquad \mathcal{L}_{\text{CE}} = -\frac{1}{M} \sum_j \sum_i Y_{ij} \log P_{ij}, \qquad \mathcal{L}_{\text{MinSum}} = \frac{1}{M} \sum_{i,j} P_{ij} D_{ij},$$

with $\lambda = 0.1$. The cross-entropy term is per-goal imitation of the solver and does most of the work. The min-sum term is a mild geometric regularizer that penalizes assigning distant goals and prevents degenerate near-uniform predictions early in training. It is only a first-hop proxy, with no tour chaining and no collision term, so the true mTSP objective reaches training solely through the labels Y.

Training directly on the solver's true post-allocation cost would target the objective we actually measure, but it is impractical at these scales. The cost is only defined once a discrete

allocation has been decoded from P and CBS has planned around the conflicts, so as a function of the network's output it is a step function: small changes in P leave it unchanged until an arg-max flips, and then it jumps. Gradient descent has nothing to follow. Estimators that do not need a gradient, policy-gradient methods among them, exist and would apply, but they need many cost evaluations per update, and every evaluation is a CBS solve costing seconds to a minute at the scales of Section 6. Labels, by contrast, are generated once and then reused across all 50 epochs. Imitating the solver's allocation recovers most of the signal for a small fraction of that compute, which is why the collision-aware alternatives are left to future work.

3.5 Data generation and deployment

Each sample is produced by placing agents and goals on a map, computing D and G by BFS, and calling the solver for the optimal Y. Instances with unreachable goals are rejected, as are those that exceed a CBS node budget of 50k expansions. That second filter addresses a failure mode found at high wall density, where an instance passes the reachability check but CBS escalates constraints without ever terminating.

At deployment the arg-max allocation is decoded, each agent's visit order is fixed, and CBS plans the paths. Ordering is done by exhaustive enumeration: for an agent holding k goals we evaluate all $k!$ orderings and keep the cheapest, which is optimal by construction. Beyond eight goals per agent the enumeration becomes impractical, so we fall back to nearest-neighbor construction followed by 2-opt improvement. When an allocation admits no collision-free plan, the next candidate in a probability-ranked chain is tried, taken best-first over the joint probability $\prod_j P_{a_j,j}$ up to $k = 3$. This is the learned counterpart of the solver's k-best escape.

4 Experimental Setup and Metrics

Experiments run on procedurally generated grids and on the four 32×32 benchmark maps that RobustMCPF itself uses (Figure 5). Only `maze` and `room` have genuine corridor and room structure, while `empty` and `random` are unstructured. That distinction drives several of the results below.

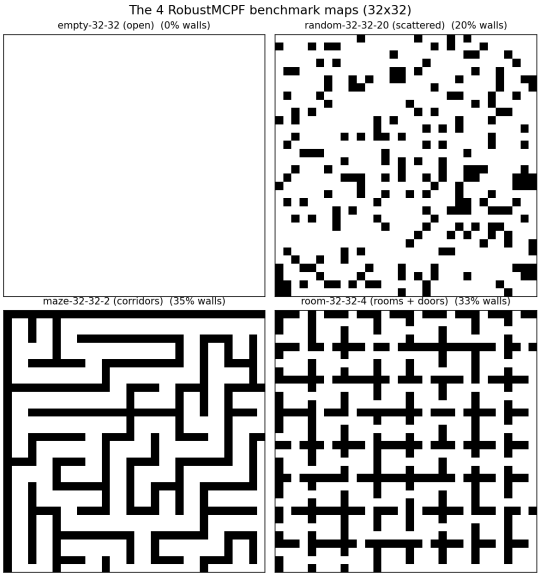


Figure 5: The four 32×32 benchmark maps: `empty`, `random-32-32-20`, `maze-32-32-2` and `room-32-32-4`. Only the last two are structured.

Allocation-quality metrics compare predicted and ground-truth allocations directly, with no planning involved. Per-goal accuracy is the fraction of goals routed to the solver's chosen agent. It is a lenient measure, since one wrong goal out of M still scores $(M-1)/M$. Full-assignment accuracy is the all-or-nothing indicator that every column matches, and it empirically tracks

(per-goal accuracy)^M. A proxy cost ratio compares only the first-hop BFS cost of the two allocations. Because it ignores tour chaining and collisions, values below 1.0 reflect the metric's mismatch with the true objective and not the network beating the solver.

Execution-cost metrics run both pipelines end to end, where a plan's cost is the total collision-free path length in grid steps that CBS returns. Cost ratio is the per-instance $c_{\text{nn}}/c_{\text{sol}}$ averaged over instances, so 1.0 means the network matched the exact solver. Exact match is the fraction of instances whose two integer costs are identical, and it runs far higher than full-assignment accuracy because many distinct allocations are cost-equivalent ties.

Diff is the mean gap in grid steps, **speedup** is the ratio of mean single-instance wall-times on the same node, and the **infeasible** and **fallback** counters report how often the candidate chain was needed. These are the numbers that matter, because they measure the plan an agent would actually execute.

5 Design Study: What Shaped the System

Fifteen experiments preceded the final system. Instead of recounting them chronologically, we report the five findings that determined the design, each of which fixes one property described in Section 3. Table 1 collects the supporting numbers. Throughout this section N is the number of agents and M the number of goals, and differences in accuracy are quoted in percentage points: a model scoring 0.789 against one scoring 0.772 is ahead by 1.7 points.

D1. Attention wins, and increasingly so with scale. Against an MLP and a DeepSets encoder on identical data, the row/column transformer led on every metric, and its margin over the MLP grew from 1.7 points of full-assignment accuracy at N=2 to 6.7 points at N=5. Cross-goal interdependence is what widens as problems grow, and attention is what captures it. This fixes the backbone.

D2. Representation dominates everything else. Adding G lifted full-assignment accuracy by 14 to 18 points at every agent count. For comparison, doubling the training set from 10k to 20k samples bought 0.4 points, and every capacity increase we tried bought at most 1.4, with the largest configuration diverging outright. D on its own cannot express the fact that two goals are adjacent and should be bundled. G supplies precisely that, and it is worth more than architecture, data and capacity put together: about four times as much at N=2 and roughly twice as much at N=5. This fixes the input.

D3. One size-agnostic model beats a family of fixed-size models. A universal model trained jointly across fifteen (N, M) configurations matched or beat the fixed-size models on all but the easiest configuration, gaining 2.4 to 3.5 points. It also transferred upward. Evaluated on N=5, which it had never seen in training, it beat the model built specifically for N=5 by 3.5 points. Cross-configuration training acts as a regularizer and pushes the model toward the allocation principle instead of size-specific patterns. This justifies the universal architecture.

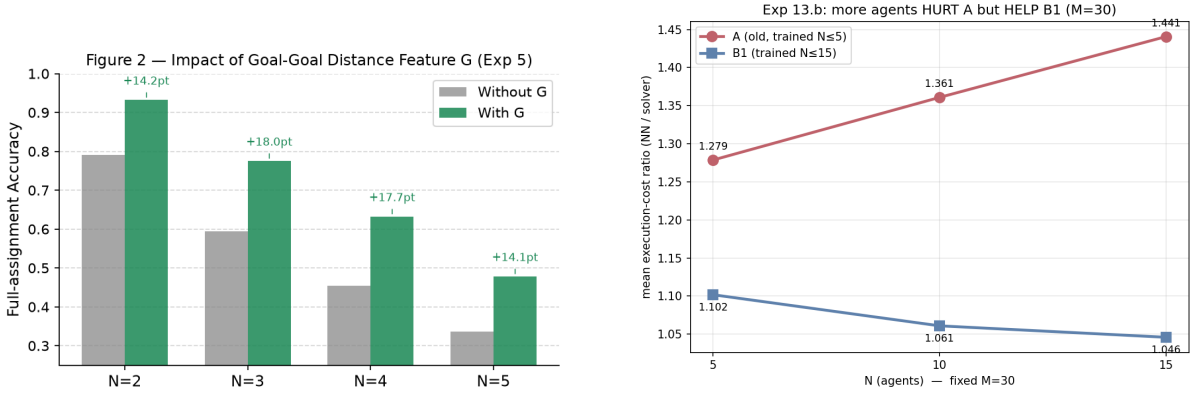
D4. Allocation accuracy is the wrong yardstick. Measured end to end, the fraction of instances whose plan costs exactly what the solver's costs (64–97%) far exceeds the fraction on which the same allocation is chosen (44–91%). Many disagreements are ties, with two agents equidistant from a goal and either choice equally good, so a model judged on allocation accuracy alone looks considerably worse than it performs. This is why every headline number in this report is an execution cost.

D5. Geometry transfers, scale does not. A model trained only on small random grids and evaluated zero-shot on the four real maps at small N, M reached a cost ratio of 1.019, meaning no meaningful degradation. Structured maps even proved easier than open ones, because walls remove the ties that an open grid leaves to chance. Pushed to large N, M, the same model collapsed to 1.246 against a map-trained model's 1.048, with a revealing signature: at M=30, adding agents hurt the model trained on $N \leq 5$ while helping the in-range model (Figure 6b). Separately, training on cheap random grids matched training on the real maps everywhere except the maze, where random obstacles never reproduce corridor structure. It is

the reason the final models are retrained at the target scale and on the target maps, and it is the direct motivation for Section 6.

Table 1: Design decisions and the evidence behind them. Effects are on full-assignment accuracy (D1–D3) or execution-cost ratio (D4–D5).

Decision	Effect	Evidence
Transformer backbone	$+1.7 \rightarrow +6.7$ pt	vs. MLP, margin growing over N=2…5
Add goal-goal distances G	$\mathbf{+14.2 \rightarrow +18.0}$ **pt**	with vs. without G, all agent counts
More data (10k → 20k)	+0.4 pt	gains already flattening at 20k
More capacity (to h256/L6)	$\leq +1.4$ pt	largest configuration diverges at lr 10^{-3}
Size-agnostic universal model	$+2.4 \rightarrow +3.5$ pt	vs. fixed-size models; +3.5 pt zero-shot at N=5
Judge by CBS execution cost	64–97% vs. 44–91%	exact-cost match vs. exact-allocation match
Train on the target map family	1.048 vs. 1.075	fixed-map vs. random; the gap is entirely the maze (1.029 vs. 1.127)
Retrain at the target scale	1.048 vs. 1.246	map-trained vs. small-scale model at large N, M
Smaller network in-distribution	1.048 vs. 1.052	h128/L6 vs. h256/L8; 2.5× the parameters bought nothing



(a) the G ablation (b) the scale-transfer scissors

Figure 6: The two findings that set up the final study. **(a)** Goal-to-goal distances lift full-assignment accuracy by 14 to 18 points at every agent count, the dominant design decision (D2). **(b)** At M=30, adding agents hurts the small-scale-trained model but helps the map-trained one, so generalization fails across problem scale, not across map geometry (D5).

Why this leads to the final study

D5 showed that a model has to be trained on structured geometry in order to handle it, since the entire advantage of map-based training over random-grid training came from the maze. If geometry-specific structure matters that much, the natural next question is whether a dedicated model per map outperforms one generalist serving all four. That is what Section 6 tests.

6 Final Study: Scale Limits and Specialists versus a Generalist

6.1 Research questions

RQ1. How far can N and M grow before the exact solver becomes impractical, on the order of a minute per instance, and what does that imply for what can be labeled and measured?

RQ2. For each model role, which training distribution works better: many instances at regular size, or fewer instances at large size?

RQ3. Do per-map specialists outperform a single generalist, in quality and in wall-time, by enough to justify routing an instance to a map-specific model?

6.2 Design

The scale wall. We sweep N and M on each map until the solver's mean wall-time approaches 60 seconds (Figure 7, interpreted under RQ1 below). This wall is what bounds the study: every training label still needs an exact solve, so it caps what a large-instance dataset can contain, and it caps how far a cost-ratio comparison can even be evaluated, since past it there is no solver baseline left to divide by.

Two training regimes. We compare two training datasets, named for what separates them. The **large dataset** covers $N \in \{60, 120, 180\}$, $M \in \{100, 225, 350\}$, with 3000 train / 500 val / 500 test instances per configuration, labeled at a per-instance solver time that on the hardest cells approaches the practical one-minute ceiling above. The **regular dataset** covers $N \in \{30, 55, 80\}$, $M \in \{50, 100, 150\}$, with 20000/2000/2000 instances per configuration, labeled in roughly ten seconds per instance, which buys 6× more samples per configuration. Five roles, one generalist trained on all four maps and four specialists trained on `empty`, `random`, `maze` and `room` respectively, are each trained once on each dataset: five **large-dataset models** (Table 2) and five **regular-dataset models** (Table 3). Ten models in total, all h128/L6.

Why only the smaller architecture. We fix the network to h128/L6 because it beat the larger h256/L8 in-distribution on every aggregate metric, with a 1.048 versus 1.052 execution-cost ratio and 0.433 versus 0.402 exact match, while running roughly 20% faster. The extra 2.5× in parameters bought no accuracy, and the larger model reached its validation optimum early and then overfitted. The ranking does reverse far out of distribution, where the larger model extrapolates better, but the large dataset places the large-instance regime inside the training distribution by construction, so the in-distribution verdict is the applicable one.

Protocol. All models see identical instances for a given (map, N, M) cell, which allows paired comparison. The dataset choice for RQ2 is made on validation data, and the specialist-versus-generalist comparison for RQ3 is reported on held-out test data, so the selection step cannot inflate the final result. The generalist is scored per map and not pooled. Without that, a uniformly mediocre generalist would be indistinguishable from one that is strong on three maps and weak on the fourth.

Table 2: Model inventory, large-dataset leg (five roles).

Model	Role	Train map(s)	N range	M range	Epochs	Batch
`tierA_joint`	generalist	all four	60–180	100–350	50	4
`tierA_empty`	specialist	empty	60–180	100–350	50	4
`tierA_random`	specialist	random	60–180	100–350	50	4
`tierA_maze`	specialist	maze	60–180	100–350	50	4
`tierA_room`	specialist	room	60–180	100–350	50	4

6.3 Results

Figure 7 does not show a clean wall. Median time grows roughly with (N, M) on `empty` and `maze`, 2 s to 52 s across the swept range, but on `random` and `room` a minority of individual instances are far slower than the median regardless of size: at N=120, M=225 on `random`, 2 of 3 instances hit the 180 s safety cap while the median at the next point tested, N=180, M=350, drops back to 36 s. Because the sweep moves N and M together rather than independently, we cannot attribute this to one or the other. What it does show is that a single pathological instance, not the typical one, is what threatens a fixed compute budget. The **labeling ceiling** is therefore map-dependent and probabilistic rather than a fixed (N, M): `maze` stayed at 0% timeouts everywhere tested, `empty` hit one 33% cell, and `random` and `room` hit 33–67% timeout rates at some cells and 0% at larger ones, which is exactly the profile that motivated the large

Table 3: Model inventory, regular-dataset leg (five roles).

Model	Role	Train map(s)	N range	M range	Epochs	Batch
`tierB_joint`	generalist	all four	30–80	50–150	50	32
`tierB_empty`	specialist	empty	30–80	50–150	50	32
`tierB_random`	specialist	random	30–80	50–150	50	32
`tierB_maze`	specialist	maze	30–80	50–150	50	32
`tierB_room`	specialist	room	30–80	50–150	50	32

Both tiers share the h128/L6 universal architecture, about 1.19M parameters (the same as Exp 12's h128/L6), lr 5×10^{-4} and gradient clipping 1.0; seed 0 is the one evaluated throughout. The large-dataset leg trained 3 seeds per role (0–2); the regular-dataset leg trained 2 (0–1, cut from a planned 3 partway through to let all four per-map configs run in parallel under the cluster's per-user GPU quota). The large-dataset batch size (4) is far smaller than the regular-dataset one (32) because its instances are larger (N, M up to 180×350) and hit GPU memory limits sooner. Best validation loss, best epoch and training wall-time are blank for the large-dataset models (only evaluation CSVs were pulled, not per-epoch logs) and partly blank for the regular-dataset models (available for the four specialists, not the joint run, whose training history spans several resubmissions and isn't attributable to one run). Full columns, including what is and is not populated per model, in `report/data/exp16/model_inventory.csv`.

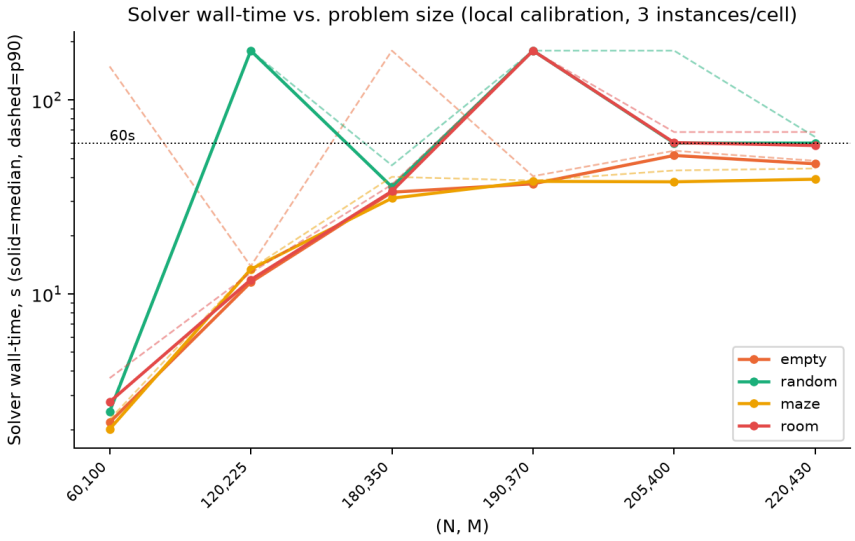


Figure 7: Solver wall-time against problem size (local calibration, 3 instances/cell, 180 s safety cap). Solid: median. Dashed: p90. The two diverge sharply on `random` and `room`, where a minority of instances are far slower than the typical one.

dataset's 500-sample-per-config size and 600 s per-instance timeout. The **evaluation ceiling** is at least N=220, M=430: every map solved all three instances there, none hit the timeout. We did not observe a size **beyond** which the solver stops returning altogether within this sweep, only rising unpredictability, so that third ceiling is not yet reached at 220×430.

Figure 8 answers RQ2 by evaluating each role's the large dataset and the regular dataset checkpoints on a common pair of grids spanning both regimes, so every checkpoint is scored both at home and away. The left panel is the regular dataset's native grid (small, $N \in \{30, 55, 80\}$, $M \in \{50, 100, 150\}$): the regular dataset is interpolating inside its training range there, while the large dataset is extrapolating downward, to sizes well below anything it ever trained on. The right panel is the large dataset's native grid (large, $N \in \{60, 120, 180\}$, $M \in \{100, 225, 350\}$): the roles reverse, the large dataset is at home and the regular dataset is extrapolating upward, to sizes beyond anything it ever trained on.

Table 4 confirms it numerically: no crossover for any role. The regular dataset wins on both its own small grid and on the large dataset's large grid, for all five roles, everywhere it was tested, including the right panel, where the regular dataset is the one extrapolating and the

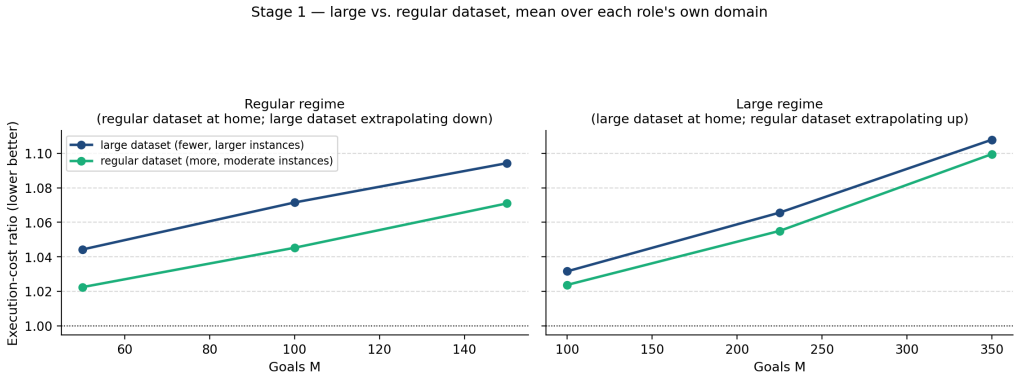


Figure 8: large vs. regular dataset, mean cost ratio over each role's own domain, on both regimes. Solid lines never cross: the regular dataset (green) sits below the large dataset (navy) at every point in both panels, whether the regular dataset is interpolating at home (left) or extrapolating up (right).

Table 4: Stage 1: large vs. regular dataset, mean cost ratio over each role's own domain (own map for a specialist, all 4 maps for the generalist), unioned across both grids (own-grid and extrapolated-grid cells both included). Winner is the lower (better) mean.

Role	Large-dataset ratio	Regular-dataset ratio	Margin	Winner
generalist	1.070	1.055	0.015	regular
empty	1.075	1.048	0.027	regular
random	1.067	1.053	0.014	regular
maze	1.056	1.045	0.011	regular
room	1.069	1.053	0.016	regular

large dataset is on native ground. The margin ranges from 0.011 on `maze` to 0.027 on `empty`, and the two curves rise together with M instead of swapping order, so the gap is consistent rather than regime-dependent. This contradicts the RQ2 hypothesis, which expected each dataset to win in its own regime. The likely explanation is sample count: the regular dataset supplies 6× more instances per configuration (20k/2k/2k against 3k/500/500), and that advantage evidently outweighs training directly at the target scale, even when the large dataset is evaluated on its own home turf. For a fixed labeling budget the result favors many regular-size instances over fewer large ones: the regular dataset's checkpoints extrapolate to the large regime better than the large dataset's checkpoints perform natively there, at a fraction of the per-instance solver cost needed to generate the labels in the first place.

Table 5: Joint (generalist) versus the native specialist, large-dataset leg, own N, M grid. Speedup is mean solver wall-time divided by mean NN pipeline wall-time, same node. n is the total instances evaluated, summed across the model's 9 (N, M) cells (joint / specialist), out of a nominal 900 (9×100); shortfalls come from whole eval cells still missing on the cluster (Table 6 in the appendix has the per-cell breakdown), not from filtered outliers.

	Joint			Specialist			
Map	Ratio	Exact	Speedup	Ratio	Exact	Speedup	n
empty	1.071	3.6%	9.1×	1.072	2.9%	16.5×	800/897
random	1.074	3.6%	8.8×	1.067	3.1%	17.1×	893/887
maze	1.057	4.0%	12.1×	1.057	3.0%	13.2×	899/898
room	1.069	3.9%	10.7×	1.066	3.8%	9.2×	786/789

Table 5 compares the joint model against each map's native specialist. The two are close everywhere. The largest gap is 0.007 on random, 1.074 for joint against 1.067 for the specialist, and maze ties to within 0.001. Neither model dominates the other on cost. The specialist is faster on three of the four maps, up to 17.1× on random against 8.8× for joint, but joint is faster on room. Exact match falls to single digits everywhere, 2.9% to 4.0%, well below the

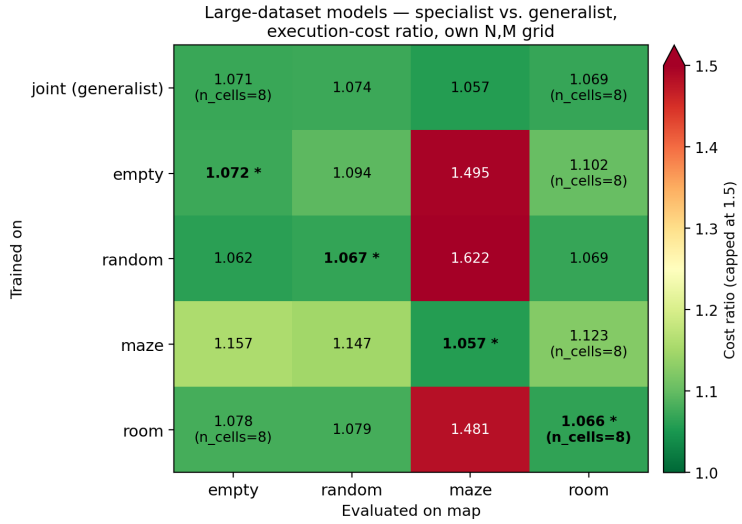


Figure 9: Full 5×4 matrix, large-dataset leg: every model (rows) on every map (columns), execution-cost ratio, own N, M grid. Asterisked cells are a specialist on its own map. A heatmap is used in place of grouped bars to keep all 20 cells, diagonal and off-diagonal, legible in one figure.

44–91% range reported at smaller scale in Section 5. Ties in optimal cost become rare as N and M grow, so cost ratio is what still discriminates between models at this size.

Figure 9 shows the full 5×4 matrix. The diagonal and the joint row sit in a narrow band, 1.057 to 1.074. The off-diagonal does not. Misrouting an instance costs little on three of the four maps, empty, random and room all stay under 1.16 when sent to each other's specialist, but is expensive on maze, where every non-maze specialist reaches 1.48 to 1.62, a 39 to 52% cost penalty over its own in-distribution number. The effect is not only allocation quality. Misrouted-to-maze cells also carry far more fallback replanning, mean NN pipeline time 7.2 to 15.2 s against 2.0 to 2.8 s in distribution, so a wrong specialist on maze is both a worse plan and a slower one to produce. This matches the maze-specific transfer gap behind D5: what fails to transfer across map geometry is maze's corridor structure specifically, whether the source model was trained on random grids (D5) or on a different real map, as here. A router only needs to be confident about one thing, whether the instance is on maze, for routing to pay off, and getting that one bit wrong is where the appeal of routing collapses.

6.4 Hypotheses

These predictions are recorded before the results are available, so that they can be confirmed or refuted rather than rationalized afterward.

On RQ2, we expect the regular dataset to win at small (N, M) and the large dataset at large (N, M), with a crossover somewhere between, since each dataset places its own regime in-distribution. The interesting quantity is not which one wins overall but where the crossover sits, because that determines how a fixed labeling budget should be spent.

On RQ3, we expect the specialist advantage to be largest on `maze` and smallest on `empty` and `random`, following D5. Training data mattered only for structured geometry, so specialization should pay only where there is structure to specialize on. If instead the specialists win uniformly across all four maps, the explanation would be capacity, with one network split across four distributions, rather than geometry. That would be a more interesting result than the one we predict.

On timing, we expect specialists and generalist to be indistinguishable, since the architecture is identical and a forward pass costs the same either way. Any measured difference should be attributed to the environment rather than the model, and one that survives scrutiny would be worth investigating.

Verdict, large-dataset leg (RQ3). The RQ3 prediction was half right. We expected the specialist advantage to be largest on maze. Instead the two tie everywhere, including maze, 1.057 against 1.057. The structure-specific effect D5 predicted is real but sits off-diagonal, not on it. It is misrouting to maze, not training on maze, that is expensive, which is the more interesting outcome we flagged as the alternative. The timing prediction failed outright: specialist and joint are not indistinguishable, speedup ranges from 2.3× for a specialist misrouted to maze to 17.1× for a specialist on its own map, and the gap tracks fallback replanning rather than the environment.

Verdict, both legs (RQ2). The RQ2 prediction failed as well, in the opposite direction from a mixed result: we expected a crossover, and found none. The regular dataset wins for every role on both regimes, so the more-data, moderate-scale strategy is the better use of a fixed labeling budget outright, not merely in its own range.

7 Discussion

When the learned allocator is the right choice. It replaces the allocation step only, so it pays off where allocation dominates and where many instances have to be solved. At large N and M, LKH's TSP grows expensive while the forward pass does not, and the gain grows with the problem: pushed to 220 agents and 430 goals on the benchmark maps, the generalist runs 6.6–16.1× faster than the solver while staying within 9.6% of optimal. Under batched, high-throughput use, allocation-only inference is orders of magnitude faster than the solver. On unstructured geometry, even cheap random-grid training transfers, so no map-specific data collection is needed. The opposite cases are equally clear. On small instances the exact solver already answers in milliseconds and CBS dominates both pipelines, leaving nothing to save. Where exactness is contractually required, an approximate allocator is inadmissible whatever its speed. And on structured maps the network has to have trained on that structure, so an unseen structured environment is a poor fit.

Limitations. Goal-visit ordering is a separate classical step in our pipeline. Exhaustive enumeration is optimal but costs $O(k!)$, and it becomes the pipeline bottleneck at seven or eight goals per agent, where the allocation itself is sub-millisecond but the step around it is not. Beyond eight goals we fall back to nearest-neighbor with 2-opt, which is fast but no longer guaranteed optimal. At that point some of the gap to the solver comes from our ordering instead of our allocation, so the attribution is no longer clean, and a learned ordering head would remove both problems at once. CBS execution cost is not differentiable, so training optimizes an imitation proxy and not the objective actually being measured. Imitation also caps quality at the teacher's level, meaning the network can match the exact solver but never beat it. Finally, the evaluation rests on one 32×32 benchmark family under basic, deterministic MAPF. Orientation, action delays, and larger or non-grid environments are untested.

8 Conclusion

A compact transformer reading two distance matrices reproduces an exact solver's goal allocation closely enough that the resulting collision-free plans cost 2% more than optimal on average in-distribution, with a single set of weights serving any number of agents and goals. The decisive design choice turned out to be representation, not architecture or scale, since goal-to-goal distances were worth more than every other improvement combined. The model generalizes across map geometry but not across problem scale, and its speed advantage appears in exactly the regimes where the exact solver's does not, at scale and in batch.

On the large-dataset leg, map specialists do not beat the generalist. The two tie on cost everywhere, including maze, and the generalist wins on wall-time for one of the four maps. What does justify routing is avoiding the alternative failure mode: sending an instance to the wrong specialist costs up to 52% on maze specifically, so a router is worth building only if it can identify the map reliably, not to chase a specialist-vs-generalist quality gap that this leg did not find.

Which training distribution, many regular-size instances or fewer large ones, makes the better use of a fixed labeling budget is now answered: the regular dataset (regular size, more samples) beats the large dataset (large size, fewer samples) outright, on its own grid and extrapolated onto the large dataset's, for every one of the five roles, with no crossover.

Future work. The solver's true tour cost is already computed during data generation and then discarded. Storing it would enable a tour-aware loss term $\sum_{j,k} \left(\sum_i P_{ij} P_{ik}\right) G_{jk}$ that directly penalizes splitting adjacent goals, targeting the failure mode that dominates at high M. A learned goal-ordering head would remove the last classical bottleneck. Approximate conflict penalties or policy-gradient signals could push training toward the true MAPF objective instead of the imitation proxy.

Acknowledgements

The RobustMCPF solver, the exact LKH and CBS pipeline used here for ground truth and as the comparison baseline, is the work of Yehonatan Kidushim, Avraham Natan, Roni Stern and Meir Kalech, and is not a contribution of this project. We thank the authors for permission to use it.

References

[1] Y. Kidushim, A. Natan, R. Stern, and M. Kalech. Robust Multiagent Combinatorial Path Finding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(35):29513–29520, 2026.

[2] G. Sharon, R. Stern, A. Felner, and N. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.

[3] K. Helsgaun. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.

[4] R. Stern et al. Multi-agent pathfinding: Definitions, variants, and benchmarks. *Symposium on Combinatorial Search (SoCS)*, 2019.

[5] N. Sturtevant. Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):144–148, 2012.

[6] H. Ma and S. Koenig. Optimal target assignment and path finding for teams of agents. *AAMAS*, 2016.

[7] O. Kaduri, E. Boyarski, and R. Stern. Algorithm selection for optimal multi-agent pathfinding. *ICAPS*, 2020.

[8] G. Sartoretti et al. PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning. *IEEE Robotics and Automation Letters*, 4(3):2378–2385, 2019.

[9] O. Vinyals, M. Fortunato, and N. Jaitly. Pointer networks. *Advances in Neural Information Processing Systems (NIPS)*, 28:2692–2700, 2015.

[10] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning. *ICLR Workshop*, 2017.

[11] W. Kool, H. van Hoof, and M. Welling. Attention, learn to solve routing problems! *ICLR*, 2019.

[12] Y. Bengio, A. Lodi, and A. Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

A Appendix

Full per-configuration results, large-dataset leg

Model	Map	N	M	Ratio	Exact	Diff mean	Speedup	n
joint	empty	60	100	1.062	2.0%	12.4	30.2×	100
joint	empty	60	225	1.119	0.0%	42.2	9.4×	100
joint	empty	60	350	1.167	0.0%	79.2	16.6×	100
joint	empty	120	100	1.025	11.0%	4.0	7.7×	100
joint	empty	120	225	1.053	0.0%	16.8	4.6×	100
joint	empty	120	350	1.092	0.0%	39.9	6.7×	100
joint	empty	180	100	1.016	15.0%	2.2	7.5×	100
joint	empty	180	225	1.035	1.0%	9.9	12.6×	100
joint	random	60	100	1.061	1.0%	12.7	6.3×	100
joint	random	60	225	1.129	0.0%	47.2	4.6×	99
joint	random	60	350	1.178	0.0%	86.0	9.3×	98
joint	random	120	100	1.029	8.1%	4.9	5.1×	99
joint	random	120	225	1.057	0.0%	18.1	7.2×	100
joint	random	120	350	1.096	0.0%	42.0	7.7×	100
joint	random	180	100	1.019	21.0%	2.7	8.7×	100
joint	random	180	225	1.034	2.0%	9.6	12.1×	99
joint	random	180	350	1.062	0.0%	25.1	16.2×	98
joint	maze	60	100	1.042	5.0%	8.8	2.1×	100
joint	maze	60	225	1.088	0.0%	30.8	12.8×	100
joint	maze	60	350	1.137	0.0%	61.0	10.5×	99
joint	maze	120	100	1.025	13.0%	4.1	10.6×	100
joint	maze	120	225	1.041	1.0%	12.2	15.1×	100
joint	maze	120	350	1.074	0.0%	29.8	17.5×	100
joint	maze	180	100	1.021	15.0%	2.8	9.6×	100
joint	maze	180	225	1.034	1.0%	9.0	14.1×	100
joint	maze	180	350	1.052	1.0%	19.6	17.4×	100
joint	room	60	100	1.054	3.1%	11.4	3.2×	96
joint	room	60	225	1.110	0.0%	39.1	15.4×	99
joint	room	60	350	1.158	0.0%	73.4	15.7×	95
joint	room	120	100	1.025	10.0%	4.0	10.1×	100
joint	room	120	225	1.050	0.0%	15.2	15.5×	98
joint	room	120	350	1.092	0.0%	38.5	9.1×	98
joint	room	180	100	1.020	18.0%	2.8	10.2×	100
joint	room	180	225	1.042	0.0%	11.5	14.5×	100
empty	empty	60	100	1.060	0.0%	11.9	42.2×	100
empty	empty	60	225	1.126	0.0%	44.4	13.7×	99
empty	empty	60	350	1.179	0.0%	84.7	19.3×	99
empty	empty	120	100	1.023	9.0%	3.7	10.4×	100
empty	empty	120	225	1.056	0.0%	17.5	14.7×	100
empty	empty	120	350	1.091	0.0%	39.6	17.6×	99
empty	empty	180	100	1.019	13.0%	2.7	10.3×	100
empty	empty	180	225	1.033	4.0%	9.3	14.1×	100
empty	empty	180	350	1.064	0.0%	26.2	16.9×	100
empty	random	60	100	1.082	1.0%	17.3	1.4×	96
empty	random	60	225	1.158	0.0%	57.7	6.3×	98
empty	random	60	350	1.206	0.0%	99.5	20.6×	94
empty	random	120	100	1.038	6.1%	6.4	1.7×	99
empty	random	120	225	1.075	0.0%	24.1	17.6×	100
empty	random	120	350	1.123	0.0%	53.9	8.1×	97
empty	random	180	100	1.027	9.0%	3.9	8.8×	100
empty	random	180	225	1.053	0.0%	15.1	5.2×	98
empty	random	180	350	1.082	0.0%	33.3	18.8×	98

Model	Map	N	M	Ratio	Exact	Diff mean	Speedup	n
empty	maze	60	100	1.387	0.0%	81.1	0.6×	92
empty	maze	60	225	1.492	0.0%	171.8	3.6×	86
empty	maze	60	350	1.532	0.0%	237.4	3.3×	74
empty	maze	120	100	1.427	0.0%	68.8	1.5×	97
empty	maze	120	225	1.496	0.0%	147.3	16.6×	89
empty	maze	120	350	1.517	0.0%	207.1	5.1×	80
empty	maze	180	100	1.487	0.0%	65.6	12.1×	98
empty	maze	180	225	1.578	0.0%	152.8	14.0×	95
empty	maze	180	350	1.540	0.0%	202.5	10.6×	74
empty	room	60	100	1.093	0.0%	19.7	2.4×	99
empty	room	60	225	1.156	0.0%	55.3	4.0×	96
empty	room	60	350	1.203	0.0%	93.9	6.1×	91
empty	room	120	100	1.043	2.0%	7.0	11.7×	100
empty	room	120	225	1.088	0.0%	26.8	6.6×	98
empty	room	120	350	1.127	0.0%	53.0	20.5×	96
empty	room	180	100	1.033	7.0%	4.6	12.5×	100
empty	room	180	225	1.070	0.0%	19.1	15.9×	100
random	empty	60	100	1.047	5.0%	9.4	43.6×	100
random	empty	60	225	1.107	0.0%	37.7	13.4×	100
random	empty	60	350	1.156	0.0%	73.7	19.6×	100
random	empty	120	100	1.020	14.0%	3.2	10.2×	100
random	empty	120	225	1.045	0.0%	14.2	14.7×	99
random	empty	120	350	1.082	0.0%	35.6	17.8×	100
random	empty	180	100	1.015	23.0%	2.1	10.0×	100
random	empty	180	225	1.028	5.0%	8.1	14.1×	100
random	empty	180	350	1.057	0.0%	23.3	17.2×	100
random	random	60	100	1.050	1.0%	10.5	12.0×	97
random	random	60	225	1.114	0.0%	41.5	15.2×	97
random	random	60	350	1.173	0.0%	83.5	20.9×	95
random	random	120	100	1.022	10.0%	3.7	11.1×	100
random	random	120	225	1.052	0.0%	16.5	16.9×	100
random	random	120	350	1.090	0.0%	39.4	18.9×	100
random	random	180	100	1.017	17.0%	2.4	11.4×	100
random	random	180	225	1.033	0.0%	9.5	15.6×	99
random	random	180	350	1.052	0.0%	21.1	19.2×	99
random	maze	60	100	1.518	0.0%	109.1	0.6×	81
random	maze	60	225	1.583	0.0%	203.1	1.5×	73
random	maze	60	350	1.615	0.0%	273.7	1.3×	33
random	maze	120	100	1.577	0.0%	92.7	1.6×	88
random	maze	120	225	1.636	0.0%	189.5	3.3×	58
random	maze	120	350	1.599	0.0%	240.2	2.2×	28
random	maze	180	100	1.691	0.0%	93.2	4.3×	95
random	maze	180	225	1.726	0.0%	192.8	4.1×	38
random	maze	180	350	1.650	0.0%	243.2	11.8×	6
random	room	60	100	1.051	1.0%	10.8	4.9×	98
random	room	60	225	1.110	0.0%	39.1	8.4×	98
random	room	60	350	1.156	0.0%	72.4	22.0×	94
random	room	120	100	1.023	12.0%	3.7	12.4×	100
random	room	120	225	1.055	0.0%	16.8	17.9×	99
random	room	120	350	1.090	0.0%	37.4	20.4×	98
random	room	180	100	1.018	14.0%	2.5	12.3×	100
random	room	180	225	1.043	0.0%	11.8	16.4×	98
random	room	180	350	1.073	0.0%	28.4	19.1×	99
maze	empty	60	100	1.142	0.0%	28.2	42.7×	100
maze	empty	60	225	1.229	0.0%	81.0	13.2×	100
maze	empty	60	350	1.268	0.0%	126.5	19.3×	100

Model	Map	N	M	Ratio	Exact	Diff mean	Speedup	n
maze	empty	120	100	1.076	0.0%	12.3	10.1×	100
maze	empty	120	225	1.153	0.0%	48.0	14.7×	100
maze	empty	120	350	1.203	0.0%	88.1	17.9×	100
maze	empty	180	100	1.055	2.0%	7.8	10.0×	100
maze	empty	180	225	1.116	0.0%	32.9	13.9×	100
maze	empty	180	350	1.174	0.0%	70.7	16.6×	100
maze	random	60	100	1.138	0.0%	28.8	12.2×	100
maze	random	60	225	1.227	0.0%	82.8	15.3×	100
maze	random	60	350	1.267	0.0%	128.8	20.4×	93
maze	random	120	100	1.062	1.0%	10.4	11.2×	100
maze	random	120	225	1.137	0.0%	43.9	16.9×	99
maze	random	120	350	1.190	0.0%	83.4	19.0×	100
maze	random	180	100	1.046	0.0%	6.6	11.4×	100
maze	random	180	225	1.103	0.0%	29.6	15.2×	99
maze	random	180	350	1.149	0.0%	60.6	18.2×	100
maze	maze	60	100	1.041	2.0%	8.7	20.3×	99
maze	maze	60	225	1.093	0.0%	32.3	2.6×	100
maze	maze	60	350	1.134	0.0%	59.5	21.4×	99
maze	maze	120	100	1.020	9.0%	3.3	12.8×	100
maze	maze	120	225	1.042	0.0%	12.7	18.4×	100
maze	maze	120	350	1.076	0.0%	30.5	21.1×	100
maze	maze	180	100	1.019	16.0%	2.5	9.4×	100
maze	maze	180	225	1.034	0.0%	9.0	12.1×	100
maze	maze	180	350	1.057	0.0%	21.5	19.9×	100
maze	room	60	100	1.121	0.0%	25.6	5.2×	99
maze	room	60	225	1.188	0.0%	66.6	1.7×	98
maze	room	60	350	1.218	0.0%	101.2	18.4×	98
maze	room	120	100	1.062	0.0%	10.0	11.7×	100
maze	room	120	225	1.108	0.0%	32.9	6.9×	99
maze	room	120	350	1.148	0.0%	62.0	16.7×	99
maze	room	180	100	1.047	0.0%	6.5	4.6×	100
maze	room	180	225	1.092	0.0%	25.2	13.4×	100
room	empty	60	100	1.060	0.0%	12.0	36.7×	100
room	empty	60	225	1.126	0.0%	44.6	12.4×	100
room	empty	60	350	1.174	0.0%	82.3	13.1×	100
room	empty	120	100	1.031	5.0%	5.0	7.7×	100
room	empty	120	225	1.066	0.0%	20.9	11.7×	100
room	empty	120	350	1.106	0.0%	45.9	6.5×	100
room	empty	180	100	1.021	13.0%	3.0	9.5×	100
room	empty	180	225	1.042	0.0%	11.9	5.3×	100
room	random	60	100	1.062	0.0%	13.0	11.2×	100
room	random	60	225	1.131	0.0%	47.8	12.1×	98
room	random	60	350	1.187	0.0%	90.0	6.0×	100
room	random	120	100	1.025	8.0%	4.2	4.8×	100
room	random	120	225	1.060	0.0%	19.1	14.4×	100
room	random	120	350	1.108	0.0%	47.5	7.1×	99
room	random	180	100	1.020	18.0%	2.9	4.5×	100
room	random	180	225	1.043	0.0%	12.3	5.6×	100
room	random	180	350	1.073	0.0%	29.5	6.4×	99
room	maze	60	100	1.470	0.0%	98.8	0.9×	90
room	maze	60	225	1.560	0.0%	195.4	0.7×	82
room	maze	60	350	1.557	0.0%	247.9	0.9×	55
room	maze	120	100	1.387	0.0%	62.3	10.0×	95
room	maze	120	225	1.474	0.0%	140.9	3.1×	80
room	maze	120	350	1.497	0.0%	199.7	13.2×	24
room	maze	180	100	1.403	0.0%	54.6	8.4×	97

Model	Map	N	M	Ratio	Exact	Diff mean	Speedup	n
`room`	maze	180	225	1.488	0.0%	129.3	2.8×	72
`room`	maze	180	350	1.490	0.0%	183.6	4.8×	12
`room`	room	60	100	1.049	1.0%	10.5	1.2×	99
`room`	room	60	225	1.105	0.0%	37.4	11.8×	96
`room`	room	60	350	1.151	0.0%	69.9	18.5×	97
`room`	room	120	100	1.021	14.0%	3.4	9.6×	100
`room`	room	120	225	1.052	0.0%	15.8	14.5×	100
`room`	room	120	350	1.089	0.0%	37.1	18.0×	97
`room`	room	180	100	1.018	15.0%	2.5	9.7×	100
`room`	room	180	225	1.044	0.0%	12.1	14.4×	100

Table 6: Full per-configuration results, large-dataset models (own N, M grid). 174 of 180 possible (model, map, N, M) cells; missing cells were not yet finished on the cluster at pull time. n: instances evaluated in that one cell (out of a nominal 100).

Reproducibility

All ten models (both tiers): h128/L6 universal transformer, `use_goal_dists` enabled, lr 5×10^{-4}, gradient clip 1.0, 50-epoch budget, seed 0 evaluated. The large-dataset models used batch size 4 and 3 seeds trained (0–2); the regular-dataset models used batch size 32 and 2 seeds trained (0–1, cut from a planned 3 partway through to let all four per-map configs run in parallel under the cluster's per-user GPU quota). The large-dataset instances (N, M up to 180×350) hit GPU memory limits at a much smaller batch size. Training ran on the cluster's `gpu` partition, `rtx_6000` GPUs. Evaluation (`full_pipeline_eval.py`, both the NN pipeline and the solver baseline) ran on the `cpu` partition, same node per cell. Torch version and exact Slurm job identifiers were not recorded on the machine this report is written on and are not yet available.

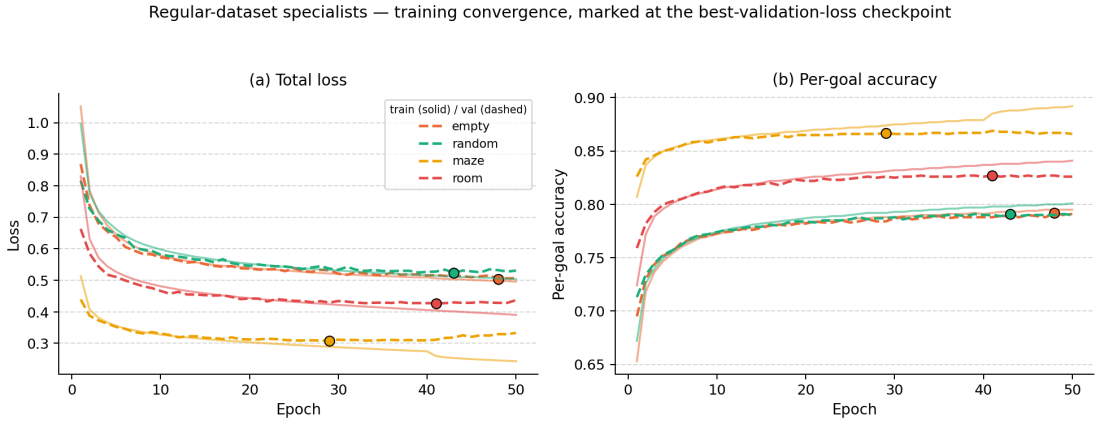


Figure 10: Training convergence for the regular-dataset specialists, the only 4 of the 10 models whose per-epoch logs were pulled from the cluster. Circles mark the best-validation-loss checkpoint. The signature is milder than the earlier large-model run (Section 5): `empty` and `random` are still improving at the 50-epoch budget's end (best epoch 48 and 43), while `maze` and `room` peak earlier (epoch 29 and 41) and drift upward afterward, so best-validation checkpointing still does some work but is not doing heavy lifting here the way it did in Section 5. The large-dataset logs and the regular-dataset generalist were not pulled for this report.

Raw data

Per-instance CSVs, aggregates and the model inventory live under `report/data/exp16/`, tracked in the repository (unlike `results/`, which is git-ignored). Per-instance columns: `inst_seed, cost_nn, cost_solver, nn_k, solver_k, conflicts_nn, conflicts_solver, alloc_ms, nn_plan_ms, solver_ms`, written by `evaluation/full_pipeline_eval.py`. `report/data/exp16/manifest.md` documents what is and is not included and the remaining open items (offline accuracy, training

logs in the requested per-epoch format, and cluster job identifiers). The raw files and the manifest keep the original run names: `tierA_*` holds the large-dataset models and `tierB_*` the regular-dataset ones. This report uses the descriptive names throughout, and the model identifiers in Tables 2 and 3 are the literal directory names, so they can be matched to the files directly.